

Lab 1-1

832204116 张林奕涵

1. Bubble sort works by iterating over the list multiple times. During each iteration, it compares each pair of ~~adj~~ adjacent elements and swaps them if they are in the wrong order. This way, the largest element moves to ~~the~~ its correct ~~to~~ position at the end of the list after each iteration. The process is repeated until no swaps are needed.
2. Insertion sort works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of ~~the~~ the list. Splitting the list into two groups: the sorted list and the unsorted list. Then pick an element from unsorted list and put it in the right place in the sorted list.
3. Merge sort uses divide and conquer idea. Firstly, dividing the list recursively into two halves until it can no more be divided. Secondly, each subarray is sorted ~~individually~~ individually using the merge sort algorithm. Finally, the sorted subarrays are merged back together in sorted order.

Code

832204116 Zhang Linyihan

```
1. import random
2. import argparse
3.
4. class Counter:
5.     def __init__(self):
6.         self.comps=0
7.     def lt(self,a,b):
8.         self.comps+=1
9.         return a<b
10.
11.     def le(self,a,b):
12.         self.comps+=1
13.         return a<=b
14.
15. def bubble_sort(a, c:Counter):
16.     arr=a[:]
17.     n=len(arr)
18.     for i in range(n-1):
19.         for j in range(n-i-1):
20.             if c.lt(arr[j+1],arr[j]):
21.                 arr[j],arr[j+1]=arr[j+1],arr[j]
22.     return arr
23.
24. def insertion_sort(a,c:Counter,count_fail_check=False):
25.     arr=a[:]
26.     n=len(arr)
27.     for i in range(1,n):
28.         key=arr[i]
29.         j=i-1
30.         while j>=0 and c.lt(key,arr[j]):
31.             arr[j+1]=arr[j]
32.             j-=1
33.         if count_fail_check and j>=0:
34.             c.comps+=1
35.
36.         arr[j+1]=key
```

```

37.     return arr
38.
39. def merge_sorting(a,c:Counter):
40.     def merge(left, right):
41.         i=j=0
42.         out=[]
43.         while i<len(left) and j<len(right):
44.             if c.le(left[i],right[j]):
45.                 out.append(left[i])
46.                 i+=1
47.             else:
48.                 out.append(right[j])
49.                 j+=1
50.             if i<len(left):
51.                 out.extend(left[i:])
52.             if j<len(right):
53.                 out.extend(right[j:])
54.
55.         return out
56.     n=len(a)
57.     if n<=1:
58.         return a[:]
59.     mid=n//2
60.     left=merge_sorting(a[:mid],c)
61.     right=merge_sorting(a[mid:],c)
62.     return merge(left, right)
63.
64.
65. def print_result(name, n, before, after, comps):
66.     print(f"== {name} ==")
67.     print(f"n= {n} ")
68.     if n <= 20:
69.         print("The array before sorting:", " ".join(map(str, before)))
70.         print("The sorted array:", " ".join(map(str, after)))
71.         print(f"Number of algorithm comparisons: {comps}\n")
72.
73.
74. def main():

```

```

75. arg=argparse.ArgumentParser(description="Sorting algorithm")
76. arg.add_argument("--algorithm", choices=["bubble", "insertion", "merge", "all"],
77.                 default="all", help="which algorithm to run")
78. arg.add_argument("--n", type=int, default=100, help="array size")
79. arg.add_argument("--seed", type=int, default=42, help="random seed")
80. arg.add_argument("--mint", type=int, default=0, help="random min (inclusive)")
81. arg.add_argument("--maxt", type=int, default=2000, help="random max (inclusive)")
82. arg.add_argument("--insertion-failed-check", default=True, action="store_true",
83.                 help="count insertion's final failed comparison")
84. args = arg.parse_args()
85.
86. rand=random.Random(args.seed)
87. arr=[rand.randint(args.mint, args.maxt) for _ in range(args.n)]
88.
89. if args.algorithm in ("bubble", "all"):
90.     c = Counter()
91.     out = bubble_sort(arr, c)
92.     assert out == sorted(arr)
93.     print_result("Bubble Sort", args.n, arr, out, c.comps)
94.
95. if args.algorithm in ("insertion", "all"):
96.     c = Counter()
97.     out = insertion_sort(arr, c, count_fail_check=args.insertion_failed_check)
98.     assert out == sorted(arr)
99.     print_result("Insertion Sort", args.n, arr, out, c.comps)
100.
101. if args.algorithm in ("merge", "all"):
102.     c = Counter()
103.     out = merge_sorting(arr, c)
104.     assert out == sorted(arr)
105.     print_result("Merge Sort", args.n, arr, out, c.comps)
106.
107. if __name__ == "__main__":
108.     main()

```

Output:

```
[Running] python -u "/Users/profighted/Desktop/大四上课程PPT/计算与复杂性/作业/Lab1/lab1-1.py"
==Bubble Sort==
n=10
The array before sorting: 1309 228 51 1518 563 501 457 285 1508 209
The sorted array: 51 209 228 285 457 501 563 1309 1508 1518
Number of algorithm comparisons: 45

==Insertion Sort==
n=10
The array before sorting: 1309 228 51 1518 563 501 457 285 1508 209
The sorted array: 51 209 228 285 457 501 563 1309 1508 1518
Number of algorithm comparisons: 40

==Merge Sort==
n=10
The array before sorting: 1309 228 51 1518 563 501 457 285 1508 209
The sorted array: 51 209 228 285 457 501 563 1309 1508 1518
Number of algorithm comparisons: 24
```

```
[Running] python -u "/Users/profighted/Desktop/大四上课程PPT/计算与复杂性/作业/Lab1/lab1-1.py"
==Bubble Sort==
n=100
Number of algorithm comparisons: 4950

==Insertion Sort==
n=100
Number of algorithm comparisons: 2490

==Merge Sort==
n=100
Number of algorithm comparisons: 549
```

```
[Running] python -u "/Users/profighted/Desktop/大四上课程PPT/计算与复杂性/作业/Lab1/lab1-1.py"
==Bubble Sort==
n=1000
Number of algorithm comparisons: 499500

==Insertion Sort==
n=1000
Number of algorithm comparisons: 242964

==Merge Sort==
n=1000
Number of algorithm comparisons: 8695
```